Chapter – 4

AJAX (Asynchronous Javascript And XML)

AJAX is not a new technology, in fact, Ajax is not even really a technology at all. It is getting tremendous industry momentum and several tool kit and frameworks are emerging. AJAX is just a term to describe the process of exchanging data from a web server asynchronously through JavaScript, without refreshing the page. But at the same time, AJAX has browser incompatibility and it is supported by JavaScript, which is hard to maintain and debug.

4.1 Fundamentals of AJAX Technology

AJAX is an acronym for Asynchronous JavaScript and XML. It is a group of inter-related technologies like JavaScript, DOM, XML, HTML/XHTML, CSS, XMLHttpRequest etc.

Ajax is just a means of loading data from the server and selectively updating parts of a web page without reloading the whole page. So it is fast.

Basically, what Ajax does is make use of the browser's built-in XMLHttpRequest (XHR) object to send and receive information to and from a web server asynchronously, in the background, without blocking the page or interfering with the user's experience.

AJAX allows you to send only important information to the server not the entire page. So only valuable data from the client side is routed to the server side. It makes your application interactive and faster.

AJAX is used for creating interactive web applications. It has become so popular that you hardly find an application that doesn't use Ajax to some extent. The example of some large-scale Ajax-driven online applications are: Gmail, Google Maps, Google Docs, YouTube, Facebook, Flickr, and so many other applications.

Following are certain feature/characteristics of AJAX.

- Ajax uses XHTML for content, CSS for presentation, along with Document Object Model and JavaScript for dynamic content display.
- Conventional web applications transmit information to and from the sever using synchronous requests. It means you fill out a form, hit submit, and get directed to a new page with new information from the server.
- With AJAX, when you hit submit, JavaScript will make a request to the server, interpret the results, and update the current screen. In the purest sense, the user would never know that anything was even transmitted to the server.
- XML is commonly used as the format for receiving server data, although any format, including plain text, can be used.
- AJAX is a web browser technology independent of web server software.
- A user can continue to use the application while the client program requests information from the server in the background.

- Intuitive and natural user interaction. Clicking is not required, mouse movement is a sufficient event trigger.
- Data-driven as opposed to page-driven.

AJAX is based on the following open standards −

- Browser-based presentation using HTML and Cascading Style Sheets (CSS).
- Data is stored in XML format and fetched from the server.
- Behind-the-scenes data fetches using XMLHttpRequest objects in the browser.
- JavaScript to make everything happen.

AJAX cannot work independently. It is used in combination with other technologies to create interactive webpages. Ajax is based on HTML, CSS, JavaScript, DOM, and XML.

HTML/CSS

HTML/CSS is website markup language for defining web page layout, such as fonts style and colors. CSS allows for a clear separation of the presentation style from the content and may be changed programmatically by JavaScript.

JavaScript

JavaScript is a web scripting language. JavaScript special object XMLHttpRequest that was designed by Microsoft. XMLHttpRequest provides an easy way to retrieve data from web server without having to do full page refresh. Web page can update just part of the page without interrupting what the users are doing.

Document Object Model

Document Object Model (DOM) method provides a tree structure as a logical view of web page. It is an API for accessing and manipulating structured documents. It represents the structure of XML and HTML documents.

XML or JSON

They are used for carrying data to and from server. XML is a format for retrieve any type of data, not just XML data from the web server. However, you can use other formats such as Plain text, HTML or JSON (JavaScript Object Notation). and it supports protocols HTTP and FTP. XMLHttpRequest is used heavily in AJAX programming. It is glue for the whole AJAX operation. JavaScript object that performs asynchronous interaction with the server. JSON (Javascript Object Notation) is like XML but short and faster than XML.

Advantages:

- Speed is enhanced as there is no need to reload the page again.
- AJAX make asynchronous calls to a web server, this means client browsers avoid waiting for all the data to arrive before starting of rendering.
- Form validation can be done successfully through it.
- Bandwidth utilization – It saves memory when the data is fetched from the same page.
- More interactive.

Disadvantages:

- Ajax is dependent on Javascript. If there is some Javascript problem with the browser or in the OS, Ajax will not support.
- Ajax can be problematic in Search engines as it uses Javascript for most of its parts.
- Source code written in AJAX is easily human readable. There will be some security issues in Ajax.
- Debugging is difficult.
- Problem with browser back button when using AJAX enabled pages.

4.1.1 Difference between Synchronous and Asynchronous Web Application

Let's understand the transmission of data through the synchronous and synchronous manner over the applications.

In **Synchronous Transmission**, data is sent in form of blocks or frames. This transmission is the full duplex type. Between sender and receiver, the synchronization is compulsory. In Synchronous transmission, there is no gap present between data. It is more efficient and more reliable than asynchronous transmission to transfer the large amount of data.

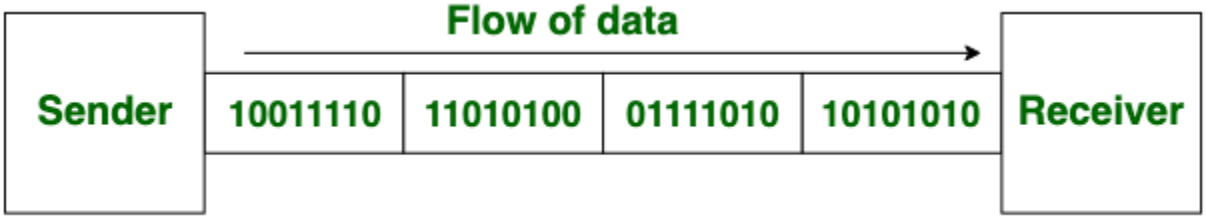

Fig: 4.1 Synchronous Transmission

In **Asynchronous Transmission**, data is sent in form of byte or character. This transmission is the half duplex type transmission. In this transmission start bits and stop bits are added with data. It does not require synchronization.

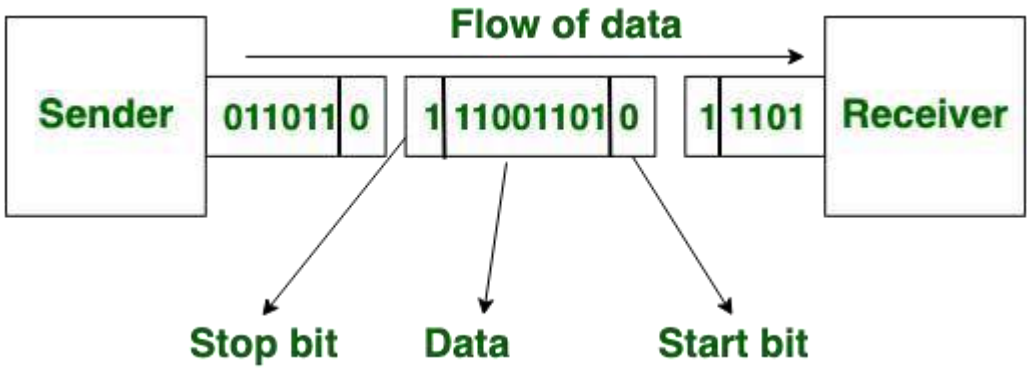

Fig: 4.2 Asynchronous Transmission

Now let's see how basic classic (Synchronous) web application and AJAX (Asynchronous) web application model differs from each other.

A **synchronous** request blocks the client until operation completes i.e. browser is unresponsive. In such case, javascript engine of the browser is blocked.

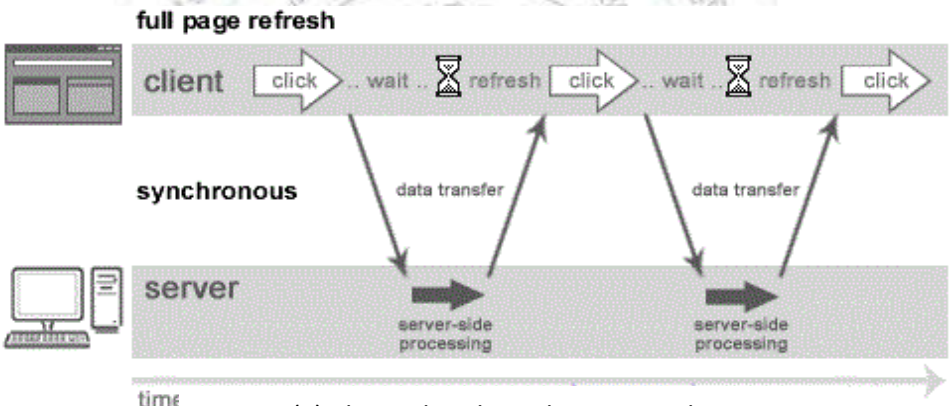

Fig: 4.3 (a) Classical Web Application working synchronous manner

As you can see in the above image, full page is refreshed at request time and user is blocked until request completes. Same can be explained as follows:

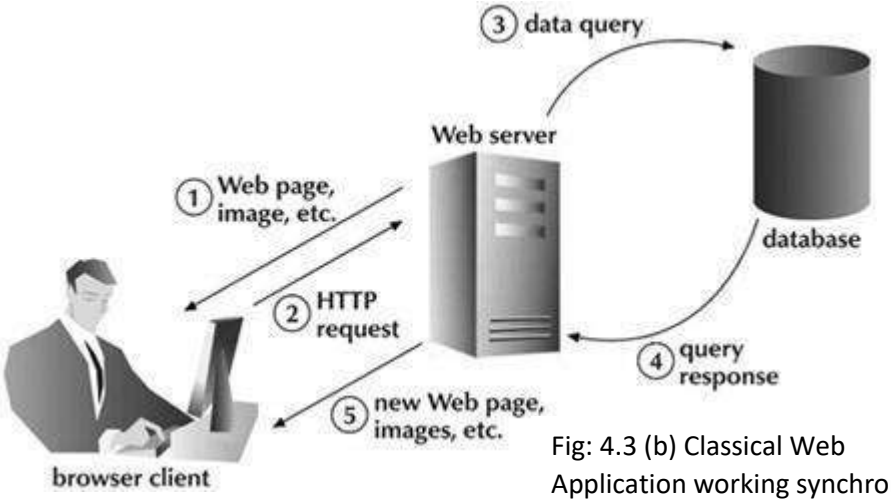

Fig: 4.3 (b) Classical Web Application working synchronous manner

In classic Web-based applications, a user input triggers a number of resource requests. Once the requests have been answered by the server, no further communication takes place until the user's next input. Such communication between client and server is known as **synchronous communication**.

Here is an explanation of classic synchronous communication passing between a browser and a Web server:

1. The user clicks a UI control in a browser-based web application.
2. The browser converts the user's action into one or more HTTP requests and passes them along to the Web-application server.
3. The application server responds to the user's requests by returning the requested data to the user. At this point the application is updated and the synchronous communication loop is complete. A new synchronous communication loop will begin when the user next clicks a UI control in their browser.

Asynchronous (AJAX Web-Application Model)

An asynchronous request doesn't block the client i.e. browser is responsive. At that time, user can perform another operation also. In such case, javascript engine of the browser is not blocked.

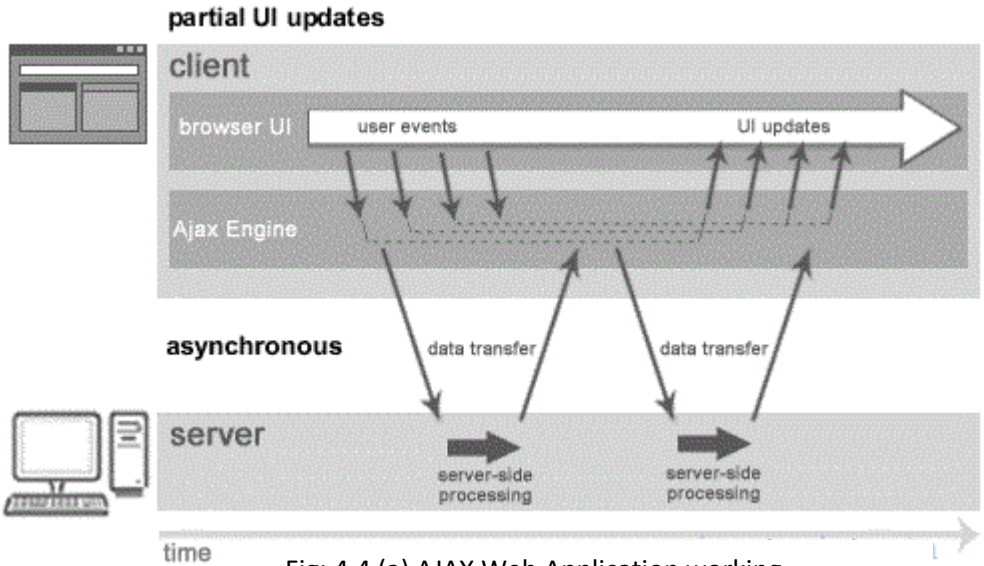

Fig: 4.4 (a) AJAX Web Application working Asynchronous manner

As you can see in the above image, full page is not refreshed at request time and user gets response from the ajax engine. Let's try to understand asynchronous communication by the image given below.

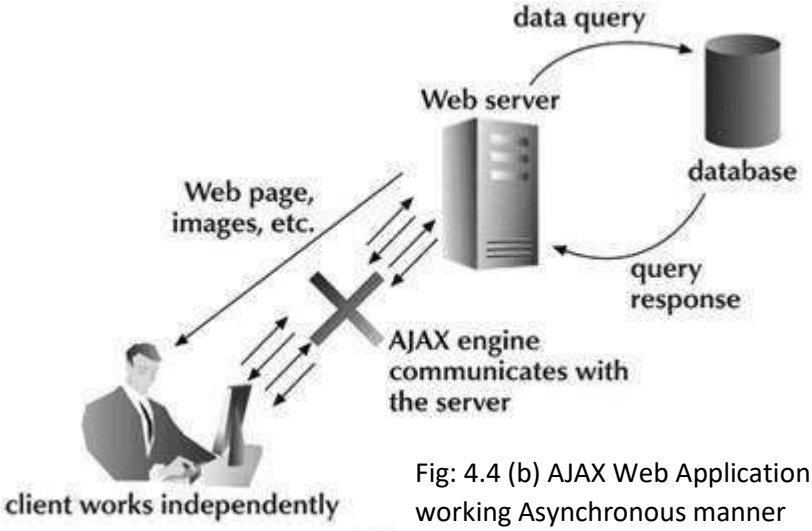

Fig: 4.4 (b) AJAX Web Application working Asynchronous manner

Here is an explanation of AJAX kind of asynchronous communication passing between a browser and a Web server:

1. User sends a request from the webpage and a javascript call goes to XMLHttpRequest object.
2. HTTP Request is sent to the server by XMLHttpRequest object.
3. Server fetch the data from the database using JSP, PHP, Servlet, ASP.net etc file.
4. Data is retrieved and view the data in webpage using HTML.

Briefly the difference can be given as follows:

Synchronous communication is limited due to the lapses in application updates that are presented to the user at regular intervals. Even if a synchronous application is designed so that it automatically refreshes information from the application server at regular intervals (for example, every 12 seconds), there will still be consistent periods of delay between data refreshes. For many applications, such update delays don't present an issue because the data they manage don't change often. Some application types however, for example stock-trading applications, rely on continuously updated information to provide optimum functionality and usability to their users.

Web 2.0 web-based applications address this issue by relying on asynchronous communication. Asynchronous applications deliver continuously updated application data to users. This is achieved by separating client requests from application updates. Multiple asynchronous communications between client and server may occur simultaneously or in parallel with one another.

While asynchronous communication delivers tremendous value to users, it presents a serious challenge to software-testing tool vendors who have difficulty emulating it with traditional test scripts.

4.1.2 XML HttpRequest technology

XMLHttpRequest object is an API that can be used by JavaScript, JScript, VBScript, and other web browser scripting languages to transfer and manipulate XML data to and from a webserver using HTTP, establishing an independent connection channel between a webpage's Client-Side and Server-Side. Mostly all browser platform support XMLHttpRequest object to make HTTP requests.

The data returned from XMLHttpRequest calls will often be provided by back-end databases. Besides XML, XMLHttpRequest can be used to fetch data in other formats, e.g. JSON or even plain text.

Using Ajax XMLHttpRequest object you can make many things easier. So many new things can't possible using HEAD request. This object allows you to making HTTP requests and receive responses from the server in the background, without requiring the user to submit the page to the server (without round trip process).

Using DOM to manipulate received data from the server and make responsive contents are added into live page without user/visual interruptions.

Using this object, you can make very user interactive web application. An object of XMLHttpRequest is used for asynchronous communication between client and server.

It performs following operations:

- Sends data from the client in the background.
- Receives the data from the server.
- Updates the webpage without reloading it.

We will see it in details in next topic.

4.2 XML HttpRequest

Since Ajax requests are usually asynchronous, execution of the script continues as soon as the Ajax request is sent, i.e. the browser will not halt the script execution until the server response comes back.

Sending Request and Receiving Response

For making the AJAX communication possible between the client and server; first of all the object of **XMLHttpRequest** must be initiated. It is possible by creating the variable with new keyword as follows in syntax:

```
var reqvariable = new XMLHttpRequest();
```

By following the above syntax, we create a variable call request1 understanding the concept here as follows:

var request1 = new XMLHttpRequest();

Now, the next step in sending the request to the server is to instantiating the newly-created request object using the open() method of the XMLHttpRequest object.

The **open()** method typically accepts two parameters— the HTTP request method to use, such as "GET", "POST", etc., and the URL to send the request to, as per the following syntax:

```
open( "method",  "URL" [ , asynchronous_flag [ , "username" [ , "password" ] ] ] )
```

Description of the parameters are as follows:

Parameter	Required?	Description
method	required	Specifies the HTTP method. Valid value GET, POST, HEAD, PUT, DELETE, OPTIONS
URL	required	Specifies URL that may be either relative parameter or absolute full URL.
asynchronous_flag	optional	Specifies whether the request should be handled asynchronously or not. Valid value TRUE, FALSE Default value FALSE TRUE means without waiting for a response, next code processing to execution queue on after the send() method. FALSE means wait for a response after the next code processing.
username	optional	Specifies username of authorize user otherwise set to null.
password	optional	Specifies password of authorize user otherwise set to null.

It migjt be written as follows for the above variable:

```
request1.open("GET","info.txt"); OR request1.open("POST","students.php");
```

The file mentioned in URL can be of any kind, like .txt or .xml, or server-side scripting files, like .php or .asp, which can perform some actions on the server (e.g. inserting or reading data from database) before sending the response back to the client.

And finally **send** the request to the server using the send() method of the XMLHttpRequest object. For the above variable it might be called as:

```
request1.send(); OR request.send(body);
```

The send() method accepts an optional *body* parameter which allow us to specify the request's body. This is primarily used for HTTP POST requests, since the HTTP GET request doesn't have a request body, just request headers.

4.2.1 Properties: (onReadyStateChange, readyState, responseText, responseXML)

An object of XMLHttpRequest is used for asynchronous communication between client and server. XMLHttpRequest (XHR) is an API that can be used by JavaScript, JScript, VBScript, and other web browser scripting languages to transfer and manipulate XML data to and from a webserver using HTTP, establishing an independent connection channel between a webpage's Client-Side and Server-Side.

The data returned from XMLHttpRequest calls will often be provided by back-end databases. Besides XML, XMLHttpRequest can be used to fetch data in other formats, e.g. JSON or even plain text.

It performs following operations:

- Sends data from the client in the background.
- Receives the data from the server.
- Updates the webpage without reloading it.

It has following properties:

Property	Description
onReadyStateChange	It is called whenever readystate attribute changes. It must not be used with synchronous requests.
readyState	represents the state of the request. It ranges from 0 to 4. • 0 UNOPENED open() is not called. After you have created the XMLHttpRequest object, but before you have called the open() method. • 1 OPENED open is called but send() is not called. After you have called the open() method, but before you have called send(). • 2 HEADERS_RECEIVED send() is called, and headers and status are available. • 3 LOADING Downloading data; responseText holds the data. After the browser has established a communication with the server, but before the server has completed the response. • 4 DONE The operation is completed fully. After the request has been completed, and the response data has been completely received from the server.
responseText	It returns the response as text

responseXML	Returns the response as XML. This property returns an XML document object, which can be examined and parsed using the W3C DOM node tree methods and properties.
Status	Returns the status as a number 200: "OK" 403: "Forbidden" 404: "Not Found"
statusText	Returns the status as a string (e.g., "Not Found" or "OK")

4.2.2 XMLHttpRequest Methods: (open(), send(), setRequestHeader())

Method	Description
abort()	Cancels the current request
getAllResponseHeaders()	Returns header information
getResponseHeader()	Returns specific header information
void open(method, URL)*	opens the request specifying get or post method and url.
void open(method, URL, async)*	same as above but specifies asynchronous or not.
void open(method, URL, async, username, password)*	same as above but specifies username and password.
void send()*	sends get request.
void send(string)*	send post request.
setRequestHeader(header,value)	it adds request headers.

Note: * indicated the methods are explained in detail in the topic 4.2 beginning.

4.3 Working of AJAX and its architecture

Working of AJAX

Ajax communicates with the server by using XMLHttpRequest Object. User send request from User Interface and JavaScript call goes to the XMLHttpRequest Object after that XMLHttp request is sent to the XMLHttpRequest Object. At that time server interacts with the database using php, servelet, ASP.net etc. The data is retrieved then the server sends data in the form of XML or Jason data to the XMLHttpRequest Callback function.

Then HTML and CSS displayed the Data on the browser. These all above process we discuss in point by point format for better understanding.

As you can see in the figure below, XMLHttpRequest object plays a important role.

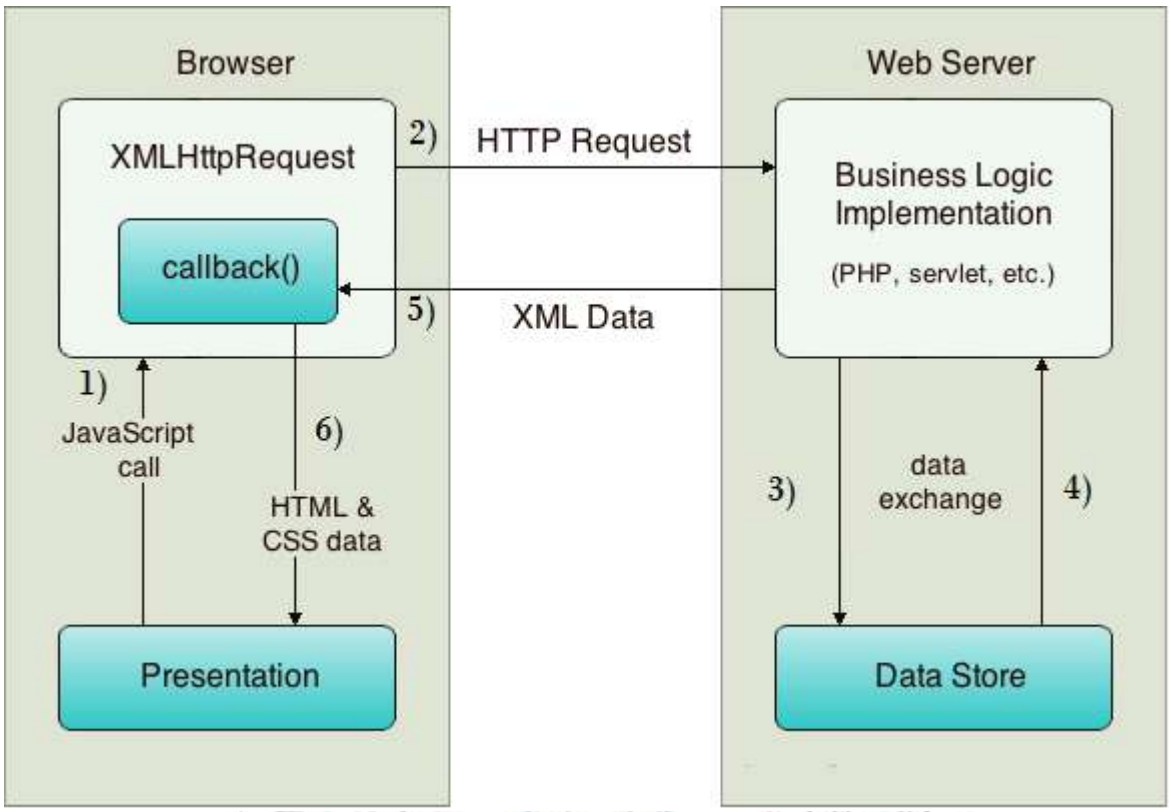

Fig: 4.5 AJAX working example

1. User sends a request from the UI and a javascript call goes to XMLHttpRequest object.
2. HTTP Request is sent to the server by XMLHttpRequest object.
3. Server interacts with the database using JSP, PHP, Servlet, ASP.net etc.
4. Data is retrieved.
5. Server sends XML data or JSON data to the XMLHttpRequest callback function.
6. HTML and CSS data is displayed on the browser.

In Ajax model, there is an Ajax engine involved in between the user and the server, which eliminates the to and fro from the user to the server and back. This Ajax engine is written in JavaScript and is in a hidden frame. It handles the user front by communicating to the user as well as handles the server front by itself. This way, the end user barely faces a waiting period.

Architecture of AJAX

- AJAX Wen application model uses JavaScript and XMLHttpRequest object for asynchronous data exchange. The JavaScript uses the XMLHttpRequest object to exchange data asynchronously over the client and server.

`

- AJAX Web application model resolve the major problem of synchronous request-response model of communication associated with classical Web application model, which keeps the user in waiting state and does not provide the best user experience.

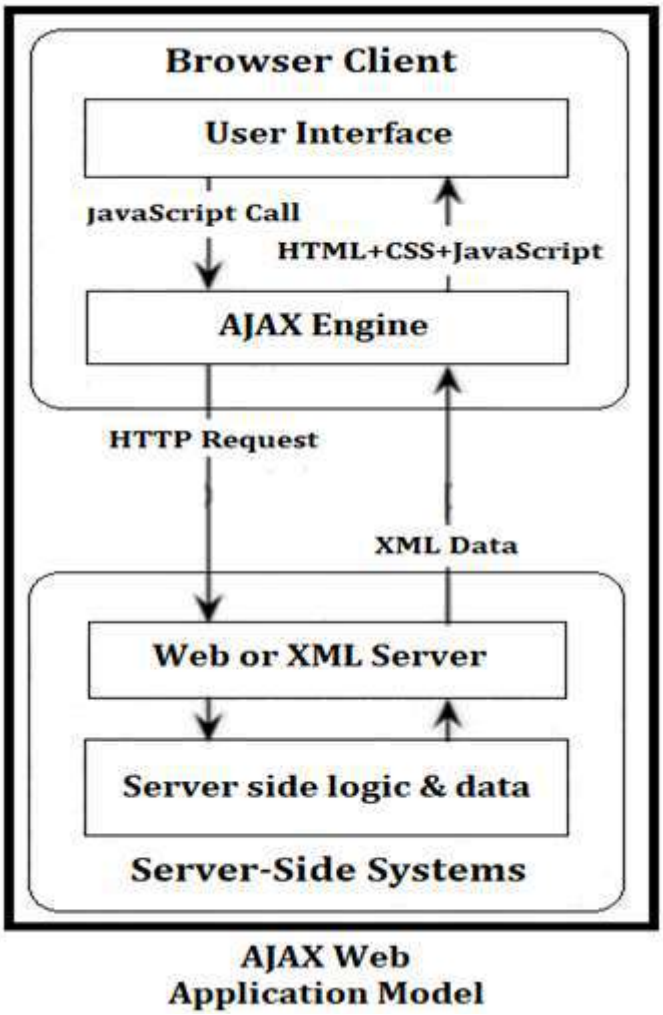

Fig: 4.6 AJAX Application Model

- AJAX, a new approach to Web applications, which is based on several technologies that help in developing applications with better user experience. It uses JavaScript and XML as the main technology for developing interactive Web applications.
- The AJAX application eliminates the start-stop-start-stop nature or the click, wait, and refresh criteria of the client-server interaction by introducing intermediary layer between the user and the Web server.
- Instead of loading the Web page at the beginning of the session, the browser loads the AJAX engine written in JavaScript.
- Every user action that normally would generate an HTTP request takes the form of a JavaScript call to the AJAX Engine.

- The server response comprises data and not the presentation, which implies that the data required by the client is provided by the server as the response, and presentation is implemented on that data with the help of markup language from Ajax engine.
- The JavaScript does not redraw all activities instead only updates the Web page dynamically.
- In JavaScript, it is possible to fill Web forms and click buttons even when the JavaScript has made a request to the Web server and the server is still working on the request in the background. When server completes its processing, code updates just the part of the page that has changed. This way client never has to wait around. That is the power of asynchronous requests.
- AJAX Engine between the client and the application, irrespective of the server, does asynchronous communication. This prevents the user from waiting for the server to complete its processing.
- The AJAX Engine takes care of displaying the UI and the interaction with the server on the user's behalf.